

Development

1. Create an authentication microservice for SSO using dotnet webapi
 - a. Inputs: username, password
 - b. Output: A token with following information: token, username, displayname, validtill
 - c. Maintains a record of tokens generated in SQL Server DB table
 - d. Following are the endpoints:
 - i. GET Api/health -- to check availability of the api
 - ii. POST Api/login -- input/output as described above
 - iii. GET Api/validate/token -- returns Ok(200) or NotFound(404)
2. Create a product microservice using dotnet webapi
 - a. Maintains a list of products and it's price
 - b. Following are the endpoints:
 - i. GET Api/health -- to check availability of the api without auth
 - ii. GET Api/product/id -- to get information about a product with auth
 - iii. GET Api/products -- to get a list of all products with auth
 - iv. PUT Api/product/id -- to update information about a product with auth and role admin
 - v. POST Api/products -- to add new products with auth and role admin
 - vi. DELETE Api/product/id -- to delete a product with auth and role admin
 - c. Information will be stored in Azure Cosmos DB using MongoDB Api
 - d. Images of products will be stored in Azure Blob Storage
 - e. Schema of product:

```
{ productid:id, productname:string, productprice:decimal, productdiscount:decimal, productcategory:string, productavailability:int, manufacturer:string, productmanufacturingdate:date, productexpirydate:date, productimageuri:string, modifiedby:string, modifiedon:datetime }
```
3. Create a UI microservice using Angular
 - a. Index.html: to show welcome page
 - b. Show products (as a list of cards showing product images) / click on any product to get more details about it
 - c. For admins: edit product information, add new product, delete a product
4. Write unit tests for all the 3 microservices : **INCOMPLETE**

DevOps

1. Containerize all the three apps and then deploy in docker using terraform docker provider. (gaining multi-container setup knowledge & IaC)
2. Use docker-compose to build following architecture:
 - a. Container 1: for sql server db
 - b. Container 2: sso
 - c. Container 3: for mongodb
 - d. Container 4: for azurite
 - e. Container 5: productheim-be
 - f. Container 6: productheim-fe
 - g. Container 7: nginx-proxy
3. Deploy all the containers using docker compose, using volumes, networks
4. Deploy all the three apps on kubernetes cluster running on local docker using minikube/kind (gaining knowledge multi-container setup in k8): **INCOMPLETE**
5. Do same activity as #2 but this time using helm charts (gaining knowledge about helm charts): **INCOMPLETE**